



**Universidad
Gerardo Barrios**



Facultad De Ciencia y Tecnología

INGENIERIA EN SISTEMAS Y REDES INFORMÁTICAS

DOCENTE:

Ing. William Alexis Montes.

MATERIA:

Programación 4

NOMBRE DE LA ACTIVIDAD:

Laboratorio 1 Tercer Cómputo Segundo Avance– Semana 16.

ESTUDIANTES:

YOLANDA ISABEL MARROQUÍN ULLOA	SMSS047424
KAREN ESMERALDA PORTILLO PORTILLO	SMSS202223
ERICK JOSUÉ RIVERA VELÁSQUEZ	SMSS147524
MOISÉS ABELINO RAMÍREZ RUBIO	SMSS217724

FECHA DE ENTREGA:

17/05/2026

ÍNDICE

1. Introducción	3
2. Requerimientos del Sistema por Módulo y Prioridad	4
2.1 Módulo de Autenticación	4
2.2 Módulo de Dashboard	4
2.3 Módulo de Fisioterapeutas	5
2.4 Módulo de Pacientes	5
2.5 Módulo de Inventario	6
2.6 Módulo de Citas	6
2.7 Módulo de Asignaciones y Sesiones de Terapia	7
2.8 Módulo de Configuración	7
3. Módulos Implementados en este Avance	7
3.1 Módulo de Autenticación ✓ Implementado	7
3.2 Módulo de Dashboard ✓ Implementado	8
3.3 Módulo de Inventario ✓ Implementado	8
3.4 Módulo de Fisioterapeutas ✓ Implementado	8
4. Módulos Pendientes de Implementar	9
4.1 Módulo de Pacientes	9
4.2 Módulo de Citas	9
4.3 Módulo de Asignaciones de Equipo	9
4.4 Módulo de Sesiones de Terapia	9
4.5 APIs CRUD del Backend	9
4.6 Aspectos a considerar al clonar el proyecto	11
4.7 Evidencias de trabajo en equipo	12

1. INTRODUCCIÓN

El presente documento corresponde al **segundo avance** del proyecto final de la materia Programación Computacional IV, denominado **FisioGest**: un Sistema de Gestión Fisioterapéutica desarrollado de forma colaborativa por el equipo de trabajo.

FisioGest es una aplicación web de arquitectura desacoplada (SPA + API REST) orientada a digitalizar y centralizar los procesos administrativos y clínicos de una clínica de fisioterapia. El sistema permite gestionar pacientes, fisioterapeutas, citas, inventario de equipos y el seguimiento del progreso terapéutico, garantizando un acceso seguro mediante roles diferenciados: Administrador fisioterapeuta y paciente.

El objetivo general del proyecto es desarrollar un sistema web integral que permita a una clínica de fisioterapia administrar de manera eficiente sus operaciones diarias, centralizando la información de pacientes, fisioterapeutas, citas, inventario y sesiones terapéuticas en una sola plataforma. De forma específica, FisioGest busca implementar un módulo de autenticación seguro con control de roles, ofrecer un dashboard administrativo con métricas en tiempo real, automatizar el registro y consulta de expedientes clínicos, controlar el inventario de equipos fisioterapéuticos con alertas de stock, gestionar la agenda de citas, y registrar la evolución clínica de los pacientes para apoyar la toma de decisiones del personal especialista.

Para este segundo avance se ha entregado lo siguiente:

Interfaces principales navegables: Login, Dashboard del administrador, módulo de Inventario y módulo de Fisioterapeutas, todas conectadas mediante Vue Router como Single Page Application.

Rutas, modelos y componentes: Configuración completa de rutas en el backend (Laravel) y frontend (Vue Router), con componentes Vue reutilizables (APPLAYOUT, LOGIN, DASHBOARD, INVENTARIO, FISIOTERAPEUTAS).

Base de datos: Esquema relacional en SQLite con 7 tablas definidas mediante migraciones de Laravel: USUARIOS, FISIOTERAPEUTAS, PACIENTES, INVENTARIO, SESIONES_TERAPIA, ASIGNACIONES_EQUIPO y CITAS.

Operaciones CRUD básicas: El módulo de Inventario y el módulo de Fisioterapeutas disponen de creación, lectura, edición y eliminación desde la interfaz frontend. La autenticación (registro, login, logout) está conectada al backend mediante la API REST con tokens Sanctum.

Los módulos pendientes (Pacientes, Citas, Asignaciones y Sesiones de Terapia) tienen su estructura de base de datos ya definida y están planificados para el avance final.

2. REQUERIMIENTOS DEL SISTEMA POR MÓDULO Y PRIORIDAD

Los requerimientos se organizan por módulo funcional y se clasifican según prioridad: **Alta** (indispensable para el funcionamiento mínimo), **Media** (importante pero no bloquea el núcleo del sistema) y **Baja** (mejora la experiencia, no crítica).

2.1 MÓDULO DE AUTENTICACIÓN

ID	Requerimiento	Prioridad
R-01	Inicio de sesión con correo y contraseña	Alta
R-02	Cierre de sesión y revocación de token Sanctum	Alta
R-03	Registro de nuevos usuarios con asignación de rol	Alta
R-04	Protección de rutas autenticadas (middleware Sanctum)	Alta
R-05	Recuperación de contraseña por correo	Baja

2.2 MÓDULO DE DASHBOARD

ID	Requerimiento	Prioridad
R-06	Mostrar conteo de citas del día actual	Alta
R-07	Mostrar conteo de pacientes nuevos en el mes	Alta
R-08	Mostrar alertas de inventario con stock bajo	Media
R-09	Gráfica mensual de citas y pacientes (Chart.js)	Media
R-10	Acceso diferenciado por rol (admin vs fisioterapeuta)	Media

2.3 MÓDULO DE FISIOTERAPEUTAS

ID	Requerimiento	Prioridad
R-11	Listar todos los fisioterapeutas registrados	Alta
R-12	Crear nuevo registro de fisioterapeuta	Alta
R-13	Editar datos del fisioterapeuta	Alta
R-14	Activar / desactivar portal del fisioterapeuta	Media
R-15	Ver horario semanal del fisioterapeuta	Media
R-16	Eliminar fisioterapeuta del sistema	Alta
R-17	Filtrar por especialidad y estado	Baja

2.4 MÓDULO DE PACIENTES

ID	Requerimiento	Prioridad
R-18	Listar pacientes con datos básicos (nombre, diagnóstico, estado)	Alta
R-19	Registrar nuevo paciente con expediente clínico	Alta
R-20	Editar información del paciente	Alta
R-21	Asignar fisioterapeuta responsable al paciente	Alta
R-22	Activar acceso al portal móvil del paciente	Media
R-23	Ver historial clínico completo del paciente	Media

2.5 MÓDULO DE INVENTARIO

ID	Requerimiento	Prioridad	
R-24	Listar equipos con tipo, cantidad y estado	Alta	
R-25	Registrar nuevo equipo con nombre, modelo y categoría	Alta	
R-26	Editar información y cantidad de equipo existente	Alta	
R-27	Mostrar alertas de stock bajo (cantidad $\leq$ umbral)	Media	
R-28	Asignar equipo a un paciente (módulo asignaciones)	Media	
R-29	Filtrar por tipo y estado del equipo	Baja	

2.6 MÓDULO DE CITAS

ID	Requerimiento	Prioridad	
R-30	Listar citas con fecha, paciente, fisioterapeuta y estado	Alta	
R-31	Programar nueva cita con asignación de horario	Alta	
R-32	Cancelar o reprogramar cita existente	Alta	
R-33	Cambiar estado de cita (programada $\rightarrow$ atendida)	Media	
R-34	Vista de calendario semanal/mensual de citas	Baja	

2.7 MÓDULO DE ASIGNACIONES Y SESIONES DE TERAPIA

ID	Requerimiento	Prioridad
R-35	Registrar asignación de equipo a paciente con fecha	Media
R-36	Registrar devolución de equipo y actualizar stock	Media
R-37	Registrar sesión de terapia con evolución y observaciones	Media
R-38	Ver historial de sesiones por paciente	Media
R-39	Gráfica de evolución clínica del paciente	Baja

2.8 MÓDULO DE CONFIGURACIÓN

ID	Requerimiento	Prioridad
R-40	Gestión de usuarios del sistema (crear, editar roles)	Media
R-41	Configuración de datos de la clínica	Baja
R-42	Umbral personalizable de alerta de stock bajo	Baja

3. MÓDULOS IMPLEMENTADOS EN ESTE AVANCE

Los siguientes módulos han sido desarrollados e integrados funcionalmente entre el frontend (Vue 3) y el backend (Laravel 12) para este segundo avance.

3.1 MÓDULO DE AUTENTICACIÓN ✓ IMPLEMENTADO

Sistema de login/logout integrado con Laravel Sanctum para autenticación por token. El componente **Login.vue** valida credenciales contra la API y redirige al Dashboard mediante Vue Router.

- Endpoint POST /api/login — devuelve token de acceso
- Endpoint POST /api/logout — revoca el token activo

- Endpoint POST /api/register — crea usuario con rol asignado
- Middleware auth:sanctum protege las rutas privadas
- Ruta SPA con fallback en routes/web.php para navegación en Vue Router

3.2 MÓDULO DE DASHBOARD ✓ IMPLEMENTADO

Panel principal de administración con métricas en tiempo real y visualización gráfica. El componente **Dashboard.vue** consume los servicios de la API mediante Promise.allSettled para mostrar datos sin bloquear la interfaz ante errores.

- Tarjeta de *Citas de Hoy*: filtra citas por la fecha actual
- Tarjeta de *Pacientes Nuevos del Mes*: cuenta registros del período
- Tarjeta de *Alerta de Inventario*: muestra ítems con cantidad ≤ 5
- Gráfica de líneas mensual (Chart.js + vue-chartjs) para citas y pacientes
- Layout persistente con barra lateral (*AppLayout.vue*) y header con perfil de usuario

3.3 MÓDULO DE INVENTARIO ✓ IMPLEMENTADO

Gestión completa de equipos fisioterapéuticos. El componente **Inventario.vue** presenta 4 tarjetas de estadísticas y una tabla con CRUD funcional desde el frontend.

- Estadísticas: Total Items, En Uso, Stock Bajo, Alertas Críticas (cantidad ≤ 3)
- Tabla de equipos: nombre, tipo, cantidad, estado (Available / Stock Bajo / En Uso)
- Modal *Añadir Nuevo Equipo*: campos Nombre, Modelo/Marca, Categoría, Cantidad, Estado
- Edición en línea mediante botón *Edit* que reutiliza el mismo modal con datos precargados
- Botón *Assign* para asignación de equipo a paciente (base preparada para integración)
- Tabla de la base de datos: inventario (nombre, tipo, marca, modelo, estado, cantidad)

3.4 MÓDULO DE FISIOTERAPEUTAS ✓ IMPLEMENTADO

Administración del equipo clínico. El componente **Fisioterapeutas.vue** muestra tarjetas de resumen, tabla con toggle switches animados y modales de registro/edición.

- Estadísticas: Total Especialistas y Activos Hoy (calculados reactivamente)
- Tabla: avatar, nombre, especialidad, toggle de estado (Activo / Inactiva), acciones

- Toggle switch CSS animado para activar/desactivar portal del especialista
- Modal *Nuevo Registro*: nombre, fecha nacimiento, especialidad, estado, correo
- Botón *Ver Horario*: modal con horario semanal (Lun–Vie 09:00–17:00)
- Tabla de la base de datos: fisioterapeutas (nombre, apellido, especialidad, telefono, activo)

4. MÓDULOS PENDIENTES DE IMPLEMENTAR

Los siguientes módulos cuentan con su estructura de base de datos ya definida mediante migraciones de Laravel pero aún no tienen componentes Vue ni controladores de API completamente desarrollados. Serán implementados en el avance final.

4.1 MÓDULO DE PACIENTES

Pantalla central para la digitalización de expedientes. Permitirá al administrador inscribir personas y gestionar su acceso al portal móvil.

- Pendiente: componente Vue, controlador *PacienteController*, rutas API CRUD
- Pendiente: vista de historial clínico del paciente

4.2 MÓDULO DE CITAS

Gestión de la agenda de citas entre pacientes y fisioterapeutas con control de estados.

- Pendiente: componente Vue con calendario y listado, controlador *CitaController*
- Pendiente: cambios de estado (programada → atendida / cancelada / reprogramada)

4.3 MÓDULO DE ASIGNACIONES DE EQUIPO

Control del préstamo y devolución de equipos del inventario hacia los pacientes.

- Pendiente: flujo de asignación desde el módulo de Inventario, actualización automática de stock

4.4 MÓDULO DE SESIONES DE TERAPIA

Registro del seguimiento clínico de cada sesión terapéutica con indicador de evolución.

- Tabla de la BD lista: *sesiones_terapia* (fecha, duracion_minutos, tipo_terapia, evolucion, observaciones)
- Pendiente: componente Vue de progreso por paciente, gráfica de evolución

4.5 APIS CRUD DEL BACKEND

El backend actualmente solo expone los endpoints de autenticación. Se deben crear los controladores API RESTful para cada entidad y conectarlos al frontend.

- Pendiente: PacienteController (index, store, update, destroy)
- Pendiente: CitaController (index, store, update, destroy)
- Pendiente: FisioterapeutaController (index, store, update, destroy)
- Pendiente: InventarioController (index, store, update, destroy)
- Pendiente: AsignacionController y SesionController

Módulo	Frontend	Backend (API)	Base de Datos
Autenticación	✓ Listo	✓ Listo	✓ Listo
Dashboard	✓ Listo	Parcial	✓ Listo
Inventario	✓ Listo	Pendiente	✓ Listo
Fisioterapeutas	✓ Listo	Pendiente	✓ Listo
Pacientes	Pendiente	Pendiente	✓ Listo
Citas	Pendiente	Pendiente	✓ Listo
Asignaciones	Pendiente	Pendiente	✓ Listo
Sesiones/Progresos	Pendiente	Pendiente	✓ Listo

4.6 ASPECTOS A CONSIDERAR AL CLONAR EL PROYECTO

Para que el proyecto pueda ejecutarse correctamente en un nuevo equipo, es necesario contar con el entorno de desarrollo configurado y seguir los pasos que se detallan a continuación. Estos pasos garantizan que tanto el backend (Laravel 12) como el frontend (Vue 3) queden completamente operativos.

Requisitos previos

- PHP 8.2 o superior con las extensiones mbstring, openssl, pdo_sqlite, fileinfo y xml habilitadas.
- Composer 2.x para la gestión de dependencias del backend.
- Node.js 18 LTS o superior y npm 9 o superior para la compilación del frontend Vue 3.
- Git instalado para clonar el repositorio del proyecto.
- Editor de código recomendado: Visual Studio Code.

1. Clonar el repositorio

Ejecutar los siguientes comandos en la terminal para obtener el código fuente:

```
git clone <url-del-repositorio>
```

```
cd fisiogest
```

2. Instalar las dependencias del backend (Laravel)

Dentro de la carpeta del proyecto, instalar las dependencias de PHP definidas en composer.json:

```
composer install
```

3. Instalar Laravel Sanctum. Dado que el sistema utiliza Sanctum para la autenticación basada en tokens, es necesario instalarlo dentro de la carpeta del proyecto mediante Composer:

```
composer require laravel/sanctum
```

4. Configurar el archivo de entorno

Duplicar el archivo .env.example y renombrarlo como .env, luego generar la clave de aplicación:

```
cp .env.example .env
```

```
php artisan key:generate
```

Verificar que en el archivo .env la conexión a base de datos esté configurada como SQLite (DB_CONNECTION=sqlite) y crear el archivo de base de datos vacío en database/database.sqlite si aún no existe.

5. Ejecutar las migraciones y los seeders

Crear las tablas de la base de datos (usuarios, fisioterapeutas, pacientes, inventario, citas, sesiones_terapia, asignaciones_equipo) y cargar los datos iniciales:

`php artisan migrate`

`php artisan db:seed`

6. Instalar las dependencias del frontend (Vue 3)

Instalar las librerías de Node.js declaradas en package.json (Vue Router, Axios, Chart.js, vue-chartjs, entre otras):

`npm install`

7. Levantar los servidores de desarrollo

Es necesario ejecutar dos procesos en paralelo, en terminales separadas dentro de la carpeta del proyecto. El primero levanta el servidor de Laravel y el segundo compila y sirve el frontend de Vue:

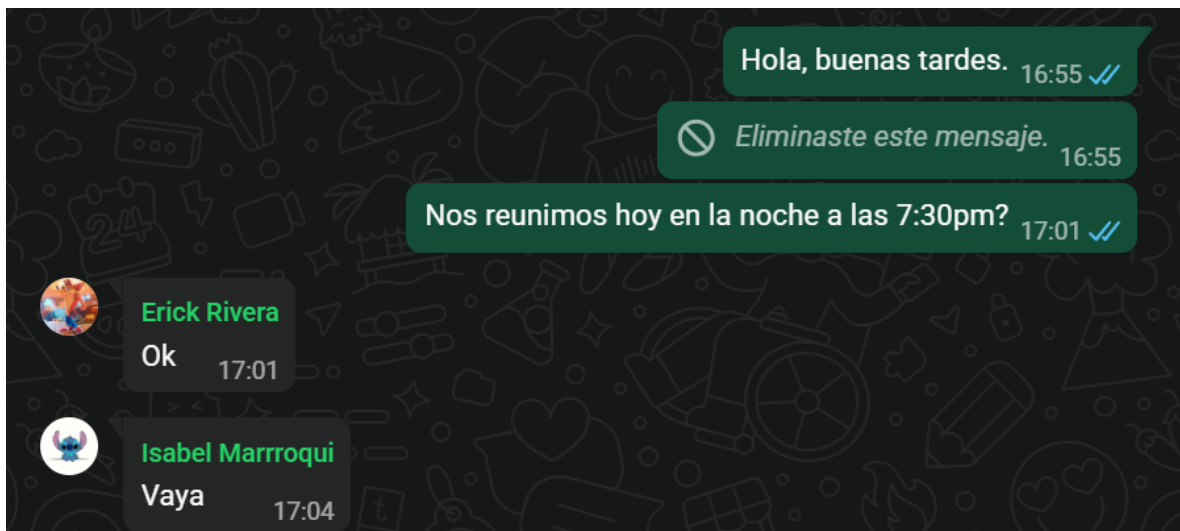
`php artisan serve`

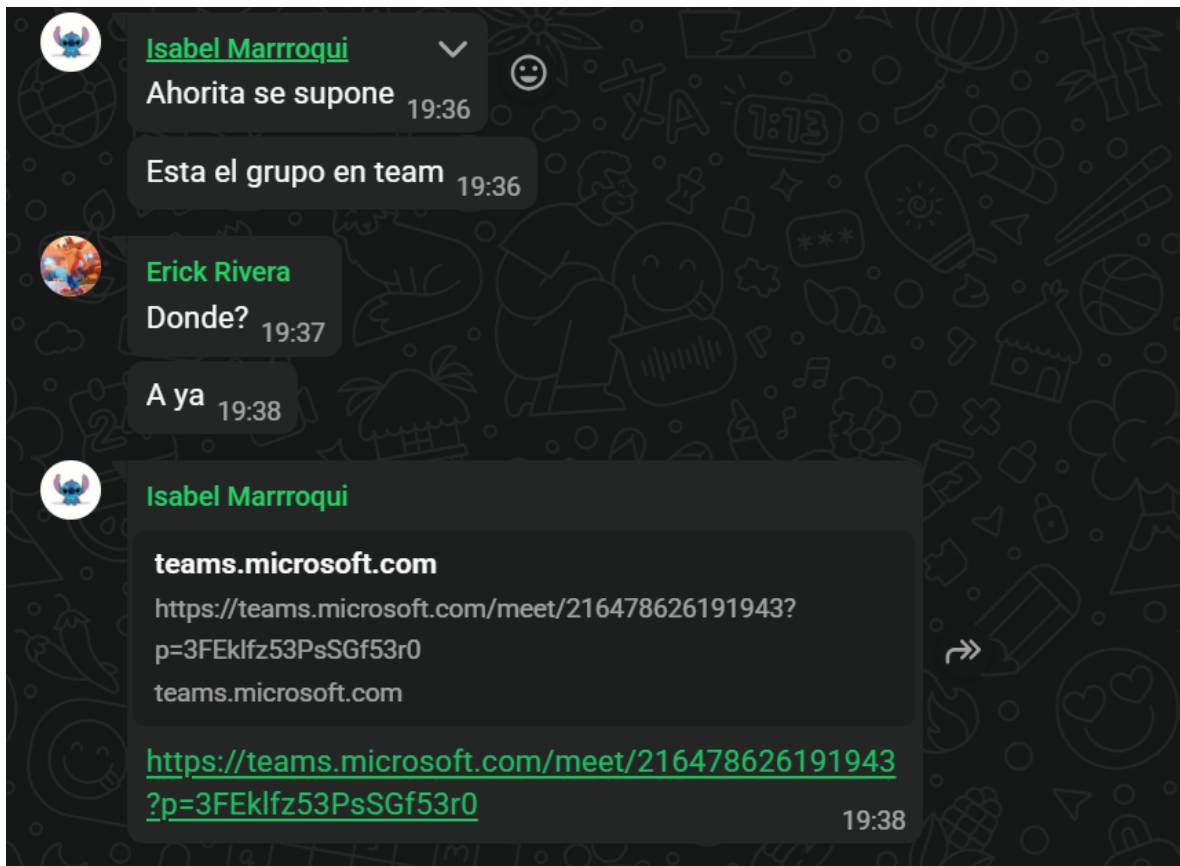
`npm run dev`

8. Acceder a la aplicación

Una vez levantados ambos servidores, abrir en el navegador la URL <http://127.0.0.1:8000>. El sistema redirigirá automáticamente al módulo de Login. Las credenciales de prueba se generan mediante los seeders ejecutados en el paso 5.

4.7 EVIDENCIAS DE TRABAJO EN EQUIPO





Universidad Gerardo Barrios

Comisión de Acreditación de la
Calidad de la Educación Superior
UNIVERSIDAD GERARDO BARRIOS (UGB)
ACREDITADA
2015-2024

Facultad De Ciencia y Tecnología

INGENIERIA EN SISTEMAS Y REDES INFORMÁTICAS

DOCENTE:

Ing. William Alexis Montes.

MATERIA:

Programación 4

NOMBRE DE LA ACTIVIDAD:

Laboratorio 1 Tercer Cómputo Segundo Avance– Semana 16.

ESTUDIANTES:

YOLANDA ISABEL MARROQUÍN ULLOA	SMSS047424
KAREN ESMERALDA PORTILLO PORTILLO	SMSS202223
ERICK JOSUÉ RIVERA VELÁSQUEZ	SMSS147524
MOISÉS ABELINO RAMÍREZ RUBIO	SMSS217724

MOISÉS ABELINO RAMÍREZ RUBIO
Definición de estilo: Normal

MOISÉS ABELINO RAMÍREZ RUBIO
Definición de estilo: Título 1; Título principal: Fuente: Negrita

Historial de versiones

Mostrar ediciones ☒ 0 de 1 ediciones

Modificado por: MOISÉS ABELINO RAMÍREZ RUBIO	23:21
Modificado por: MOISÉS ABELINO RAMÍREZ RUBIO	23:10
Modificado por: MOISÉS ABELINO RAMÍREZ RUBIO	23:00
Modificado por: YOLANDA ISABEL MARROQUÍN ULLOA	20:39
Modificado por: ERICK JOSUÉ RIVERA VELÁSQUEZ	20:22
Modificado por: ERICK JOSUÉ RIVERA VELÁSQUEZ	20:06
Modificado por: ERICK JOSUÉ RIVERA VELÁSQUEZ	19:55
Ayer, 16 de mayo de 2026	
Compartido por: YOLANDA ISABEL MARROQUÍN ULLOA	17:40
Modificado por: YOLANDA ISABEL MARROQUÍN ULLOA	17:23